

Bloom

PARFUMS

Bugs Resolved — Admin Panel

DEBUGGING REPORT — MARCH 2026

Project: Bloom Parfums

Stack: Next.js 14 + Laravel 11 + MySQL

Date: March 12, 2026

Status: All Issues Resolved

Plain-language guide to every bug encountered
and how each one was diagnosed and fixed.
Covers 5 bugs across PHP, Laravel Sanctum,
Next.js middleware, and browser cookie handling.

Bugs Resolved — Bloom Parfums Admin Panel

A plain-language guide to every bug we hit and how we fixed it. Each section answers: **What broke? Why? How we fixed it?**

Bug 1 — Dashboard showed "Failed to load dashboard data (network error)"

What happened?

The admin dashboard page loaded visually, but no data appeared. The browser console showed a network error.

Why?

The PHP built-in server (`artisan serve`) was **crashing silently** every time an authenticated request reached the server. No error log. Just — dead.

The root cause was a **circular Sanctum loop**:

```
config/sanctum.php → 'guard' => ['admins']
config/auth.php    → 'admins' guard has driver: 'sanctum'
```

When `auth:sanctum` middleware ran, Sanctum read its own config to find which guards to check. It found `admins`. The `admins` guard driver is `sanctum`. So Sanctum tried to resolve itself again — **infinite loop** → **PHP stack overflow** → **process killed**.

How we fixed it

File: `config/sanctum.php`

```
// BEFORE (causes loop)
'guard' => ['admins'],

// AFTER (correct — web is the SPA session guard, not a Sanctum guard)
'guard' => ['web'],
```

File: `routes/api.php`

```
// BEFORE (triggers the loop)
->middleware(['auth:sanctum', 'ensure.admin', 'throttle:300,1'])

// AFTER (use the specific admin guard directly)
->middleware(['auth:admins', 'ensure.admin', 'throttle:300,1'])
```

Rule to remember

*If your custom guard has `driver: sanctum`, **never** put it inside `sanctum.guard`. That config is for SPA cookie sessions only. Your API guard should be referenced directly as `auth:your-guard`.*

Bug 2 — Login worked but dashboard immediately redirected back to login (redirect loop)

What happened?

You logged in at `/admin/login`, entered correct credentials, got redirected to `/admin/dashboard` — then 2 seconds later you were back at `/admin/login`. This kept repeating forever.

Why? — Part 1: Wrong Domain

The backend set the `admin_token` cookie on its own domain (`127.0.0.1:8000`):

```
// AdminAuthController.php - cookie set on the backend
->cookie('admin_token', $token->plainTextToken, ...)
```

The Next.js middleware runs for requests to `localhost:3000`. These are **different origins**. The browser does not share cookies between them.

So the middleware always saw the cookie as missing:

```
User logs in → backend sets cookie on :8000
Middleware checks cookie on :3000 → NOT FOUND
Middleware redirects to /admin/login → loop
```

How we fixed it — Part 1

The backend now returns the token **in the JSON body** too:

```
// AdminAuthController.php
return response()->json([
  'message' => 'Authenticated.',
  'token'   => $token->plainTextToken, // ← added this
  'admin'  => ['id' => $admin->id, 'email' => $admin->email],
])
```

The frontend login service reads the token from the response and sets it as a cookie on its own domain (`localhost`):

```
// services/api.ts
login: async (payload) => {
  const { data } = await apiClient.post('/v1/admin/auth/login', payload);
  if (typeof document !== 'undefined' && data.token) {
    // Set cookie on localhost — the domain Next.js middleware can read
    document.cookie = `admin_token=${encodeURIComponent(data.token)}; path=/; max-age=86400; SameSite=Lax`;
  }
  return data;
},
```

Why? — Part 2: Race Condition

Even after the domain fix, the loop still happened sometimes. The flow was:

```
1. Login API returns token
2. JS sets document.cookie ← happens here
3. window.location.href = '/admin/dashboard' ← happens IMMEDIATELY after
4. Browser sends HTTP request to /admin/dashboard
   - but the cookie write wasn't fully committed yet
5. Middleware checks cookie → NOT FOUND → redirect to login
```

Browsers write cookies synchronously in theory, but between setting a cookie and the browser sending it in a navigation request, there can be a **1-2 frame gap**.

How we fixed it — Part 2

Added a **150ms delay** between setting the cookie and navigating:

```
// app/admin/login/page.tsx
try {
  await adminAuthService.login({ email, password });
  // Wait 150ms so the cookie is guaranteed to be in the browser's store
  // before the navigation request is sent
  setTimeout(() => {
    window.location.href = '/admin/dashboard';
  }, 150);
}
```

Also switched from `router.push()` to `window.location.href`:

Method	Type	Problem
<code>router.push('/admin/dashboard')</code>	Soft/client navigation	Next.js RSC fetch — may not carry new cookies
<code>window.location.href = '/admin/dashboard'</code>	Hard HTTP navigation	Always carries all cookies in the request

Rule to remember

When you set a cookie on the frontend and immediately navigate, **always use a hard redirect** (`window.location.href`) not a framework router push. And add a small delay (100–200ms) to avoid the cookie race condition.

Bug 3 — PHP artisan serve crashed on Windows with no error

What happened?

Running `php artisan serve` on Windows, the process died silently when an authenticated HTTP request came in. No error in `storage/logs/laravel.log`. The port just disappeared from `netstat`.

Why?

This was the Sanctum circular loop described in Bug 1. The PHP built-in server's single-threaded model means an infinite recursion kills the entire process with no recovery.

On Linux/Mac, PHP might show a segfault. On Windows it just vanishes.

How to diagnose a silent PHP crash

```
# 1. Start the server
Start-Job { Set-Location "c:\...\backend"; php artisan serve --host=127.0.0.1 --port=8000 }

# 2. Make a request
Invoke-WebRequest "http://127.0.0.1:8000/api/v1/admin/products" -Headers @{Authorization="Bearer T
OKEN"}

# 3. Immediately check if process is still alive
netstat -ano | Select-String ":8000.*LISTEN"
# If this returns nothing → PHP crashed on that request
```

Rule to remember

If `artisan serve` dies silently with no logs — it's almost always a **PHP-level crash** (infinite loop, stack overflow, out of memory). Add a public test route with no auth to isolate whether the crash is in middleware or controller logic.

Bug 4 — Apache took over port 8000

What happened?

After experimenting with Apache VirtualHost config, Apache started listening on port 8000 instead of Laravel.

Why?

The line `Listen 8000` was accidentally added to `C:\xampp\apache\conf\httpd.conf`. Apache grabbed the port before Laravel could.

How we fixed it

Removed `Listen 8000` and `Listen 8080` from Apache config. Apache now only runs on ports 80 and 443.

```
# httpd.conf - BEFORE
Listen 80
Listen 8000 ← removed
Listen 8080 ← removed

# httpd.conf - AFTER
Listen 80
```

Rule to remember

Check `httpd.conf` and `httpd-vhosts.conf` if your dev server port is suddenly "taken". Apache's `Listen` directive binds ports globally.

Bug 5 — `DecryptException` on cookie reading

What happened?

An earlier version used a middleware that read the `admin_token` cookie server-side using Laravel's `$request->cookie()`. It crashed with `DecryptException`.

Why?

Laravel's `$request->cookie()` method runs the value through `Crypt::decrypt()` automatically. But Sanctum tokens stored as cookies from API routes are **not encrypted** — they are raw plain-text tokens. Decrypting a plain-text string throws an exception.

```
// WRONG - tries to decrypt, crashes on plain-text token
$token = $request->cookie('admin_token');

// CORRECT - reads raw value, no decryption
$token = $request->cookies->get('admin_token');
```

Rule to remember

`$request->cookie('name')` decrypts the value (web route cookies are encrypted by Laravel). `$request->cookies->get('name')` reads the raw value (what you want for Sanctum API tokens).

Grammarly Console Warnings (Not a Bug)

Warning: Extra attributes from the server: data-new-gr-c-s-check-loaded

This is from the **Grammarly browser extension** injecting attributes into the HTML. Not your code. Disable Grammarly for `localhost` in the extension settings to remove the noise.

Quick Reference — How the Auth Flow Works Now

```
[Browser: localhost:3000]
|
| 1. POST /api/v1/admin/auth/login
|    { email, password }
|
▼
[Backend: 127.0.0.1:8000]
|
| 2. Returns JSON: { token: "36|abc...", admin: {...} }
|
▼
[Frontend: api.ts login()]
|
| 3. document.cookie = "admin_token=36|abc...; path=/; max-age=86400"
|    (written on localhost domain)
|
| 4. setTimeout 150ms (cookie commit guarantee)
|
| 5. window.location.href = "/admin/dashboard"
|    (hard HTTP request — browser sends cookie)
|
▼
[Next.js Middleware: middleware.ts]
|
| 6. request.cookies.get('admin_token') → FOUND ✓
|
| 7. NextResponse.next() → allow through
|
▼
[Dashboard page loads]
|
| 8. useEffect → GET /api/v1/admin/dashboard
|    Authorization: Bearer 36|abc... (set by Axios interceptor)
|
▼
[Backend: DashboardController]
|
| 9. auth:admins middleware validates token
|    Returns real data: revenue, orders, etc.
|
▼
[Dashboard renders with live data ✓]
```